

tf.compat.v1.name_scope Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/name_scope

A context manager for use when defining a Python op.

Function Signatures

```
tf.compat.v1.name_scope( name, default_name=None, values=None
) -> None

def my_op(a, b, c, name=None): with tf.name_scope(name,
"MyOp", [a, b, c]) as scope: a = tf.convert_to_tensor(a,
name="a") b = tf.convert_to_tensor(b, name="b") c =
tf.convert_to_tensor(c, name="c") # Define some computation
that uses `a`, `b`, and `c`. return foo_op(..., name=scope)
```

Code Examples

```
tf.compat.v1.name_scope(
name, default_name=None, values=None
) -> None
```

```
tf.compat.v1.name_scope(  
    name, default_name=None, values=None  
) -> None
```

```
def my_op(a, b, c, name=None):  
    with tf.name_scope(name, "MyOp", [a, b, c]) as scope:  
        a = tf.convert_to_tensor(a, name="a")  
        b = tf.convert_to_tensor(b, name="b")  
        c = tf.convert_to_tensor(c, name="c")  
        # Define some computation that uses `a`, `b`, and `c`.  
    return foo_op(..., name=scope)
```

```
def my_op(a, b, c, name=None):  
    with tf.name_scope(name, "MyOp", [a, b, c]) as scope:  
        a = tf.convert_to_tensor(a, name="a")  
        b = tf.convert_to_tensor(b, name="b")  
        c = tf.convert_to_tensor(c, name="c")  
        # Define some computation that uses `a`, `b`, and `c`.  
    return foo_op(..., name=scope)
```

```
__enter__() -> Optional[str]
```

```
__enter__() -> Optional[str]
```

```
__exit__(  
    *exc_info  
) -> Optional[bool]
```

```
__exit__(  
    *exc_info  
) -> Optional[bool]
```

Documentation

A context manager for use when defining a Python op.

This context manager validates that the given values are from the same graph, makes that graph the default graph, and pushes a name scope in that graph (see `tf.Graph.name_scope` for more details on that).

For example, to define a new Python op called `my_op`:

Raises

`## Attributes`

Methods

`## __enter__`

[View source](#)

Return self upon entering the runtime context.

`__exit__`

[View source](#)

Raise any exception triggered within the runtime context.